encoder-> hidden state-> decoder -> per token generation

Assignment13 – Deep Learning for NLP (2)

Given: July 15 Due: July 20

Problem 13.1 (Seq2Seq Translation)

1. Explain (in about 2 sentences) the key idea of seq2seq models for machine translation.

Solution: It concatenates two neural networks, one to encode the source, followed by one to decode into the target language. Inputs are fed into the encoder token-wise. When the input it processed, the decoder generates outputs one word at a time.

---

2. Explain (in about 2 sentences) the role of beam search in the decoder.

Solution: It searches through possible decodings. Rather than committing to an output word in each step, each step keeps the top k hypotheses for the output sequence.

---

multiple candidates instead of single most likely word (greedy)

the best next word is not always the one with the highest immediate probability, but the one that leads to the best overall sentence.

Problem 13.2 (Attentional Seq2Seq) (see diagram at end of pdf)

1. Explain the design and formula from input to output in an attentional seq2seq model.

Solution: The basic design is to couple an encoder with a decoder to transform input vector sequence $x_1, \ldots, x_S$ to output vector sequence $y_1, \ldots, y_T$. There is no condition $S = T$, i.e., to generate exactly one output vector y_i for every input vector x_i — it transforms the entire input sequence into an output sequence.

The encoder is an RNN (e.g., a bidirectional RNN) that processes all input vectors resulting in the sequence $H_1, \ldots, H_S$ of encoder hidden states. This happens first, and the entire state sequence is available before the decoder starts.

To compute output y_j, the decoder receives as input the previous decoder hidden state h_{j-1} and the previous output y_{j-1}. It receives a START token as y_0, and the process repeats until a STOP token is output for y_T.

It first compares h_{j-1} to all H_i, resulting in an alignment score e_{ji} for every $i = 1, \ldots, S$. There are different ways to do this, one choice (assuming the two hidden layers have the same dimension) is to use the scalar product $e_{ji} = h_{j-1} \cdot H_i$. The vector $e_j = \langle e_{j1}, \ldots, e_{jS} \rangle$ is normalized into a probability distribution, e.g., by using $a_j = \text{softmax}(e_j)$. (softmax is the operation on

letting the decoder look back at different parts of the input sequence instead of relying only on a single fixed-size vector

But instead of relying only on h_j-1, it also computes a context vector c_j using attention

alignment score for each encoder state H_i (relevant encoder position i is for producing the next output token.)

normalizing the scores w softmax (attention weights)

Attention allows the decoder to focus on different parts of the input at each step, dramatically improving translation quality.

vectors defined by $\text{softmax}(v) = \alpha(e^v)$ where $e^{\langle v_1,\ldots,v_n\rangle} := \langle e^{v_1},\ldots,e^{v_n}\rangle$.) a_j is called the attention matrix; it captures how similar each encoder state is to the current decoder state. The context vector c is defined as the sum of all H_i weighted by a_{ji}, i.e., $c_j = \Sigma_{i=1}^{S} a_{ji} \cdot H_i$.
The decoder then uses some RNN that takes inputs $y_{j-1}; c_j$ (concatenation of vectors) in hidden state h_{j-1} and produces output y_j and the next hidden state h_j.

gives the decoder a dynamic summary of the input, tailored to step j

---

Problem 13.3 (Byte Pair Encoding)
We apply the byte-pair algorithm to the following corpus:

baabc aadc acbaa

1. What is the relevance of the target token size N? What values make sense here?

Solution: BPE increases the number of tokens by successively introducing a new token for the most frequent token pair. The language has 4 characters. So we should have $N > 4$.
N is the stop condition. If N is too large (e.g., near the number of characters in the corpus), we merge the whole corpus into very few tokens, and the overhead of storing the table of new tokens becomes big. We have to find a good trade-off here.

If N is too small → we stay close to character-level encoding, which leads to long sequences and poor modeling of frequent patterns.

If N is too large → we merge too aggressively, creating long, highly specific tokens that rarely generalize and require storing a large merge table.

---

2. Apply BPE for $N = 6$.

Solution: We start with the 4 tokens $BPE(a) = 0$, $BPE(b) = 1$, $BPE(c) = 2$, $BPE(d) = 3$. We stop when we have defined N tokens, so we need 2 iterations. The most common token is aa. So we define a new token $E := aa$ and put $BPE(E) = 4$. The corpus becomes 4 because its simply the 4th token in the vocab

bEbc Edc acbE

Now the most common token is bE. So we define a new token $F := bE$ and put $BPE(bE) = BPE(baa) = 5$.
Our final tokenization is

Fbc Edc acF

count all adjacent pairs

b a a b c
a a d c
a c b a a

aa appears 3 times
ab appears 1 time
ba appears 2 times
ac appears 1 time
bc appears 1 time
dc appears 1 time

Pairs:
bE appears 2 times
Eb appears 1 time
ac appears 1 time
cb appears 1 time
bE appears again at the end
Most frequent pair: bE

Problem 13.4 (Embeddings vs. Positional Encodings)
We compare word embeddings and positional word encodings.

Transformers (and many NLP models) need two kinds of information
- What each word is (its semantic meaning)
- Where each word is in the sequence (its position)

Word embeddings handle the first.
Positional encodings handle the second

1. Explain (in about 1 sentence) the key similarity.

Solution: Both can be used to map every element w_i in a sequence of words $w_1, \dots, w_n$ to a vector. Thus, they induce functions $f : V^n \to \mathbb{R}^{dn}$ where V is the vocabulary and d is the dimension of the encoding of each word.

symbolic input (words) into numerical input (vectors).

---

2. Explain (in about 2 sentences) the key difference.

Solution: Word embeddings always use the same function $e : V \to \mathbb{R}^d$ for all words. The function f is defined component-wise as $f(w_1, \dots, w_n) = e(w_1); \dots; e(w_n)$.

It assigns the same vector to a word every time it appears, regardless of where it is in the sentence.

Positional encodings use a function $p : V \times \mathbb{N} \to \mathbb{R}^d$ that additionally takes the position of the word as input. f is then defined by $f(w_1, \dots, w_n) = p(w_1, 1); \dots, p(w_n, n)$.
A common choice is $p(w, i) = e(w) + p_i$ where e is an embedding and p_i a vector.

It assigns a vector that depends on both the word and its position.

"cat" at position 1 → different vector than "cat" at position 7.

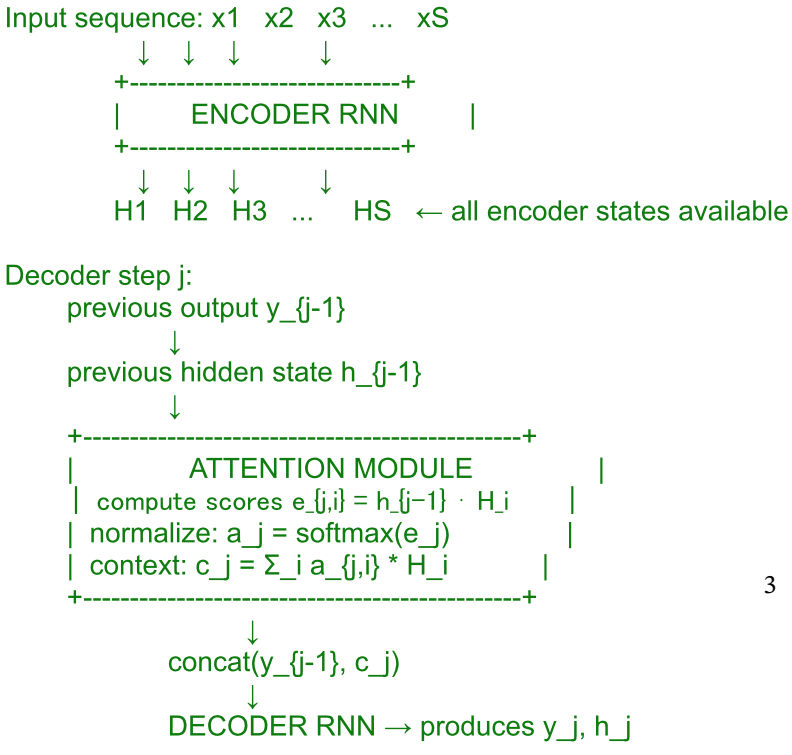
---

3. Which one can be seen as a special of the other.

Solution: A word embedding e can be seen as a positional encoding p that ignores the position, i.e., $p(w, i) = e(w)$.

Transformers have no recurrence and no convolution, so they cannot infer order on their own.

13.2



Sentence: w1 w2 w3 w4

Word embeddings:
e(w1) e(w2) e(w3) e(w4)
(same vector each time the word appears)

Positional encodings:
e(w1)+p1 e(w2)+p2 e(w3)+p3 e(w4)+p4
(position-dependent vectors)